Django ORM Optimization

select_related, prefetch_related, Prefetch

Django ORM-এ এই তিনটি ব্যবহার করা হয় database query কমানোর জন্য।
যখন model এর মধ্যে relation থাকে তখন না ব্যবহার করলে N+1 Query Problem হয়।

Relation হতে পারে:

- **ForeignKey**
- **OneToOneField**
- **ManyToManyField**

1 select_related

select_related ব্যবহার করা হয়:

- **ForeignKey**
- **OneToOneField**

এটা SQL JOIN ব্যবহার করে একটা query দিয়েই related data নিয়ে আসে।

Example Model

```
class Author(models.Model):
    name = models.CharField(max_length=100)

class Book(models.Model):
    title = models.CharField(max_length=100)
    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

Without select_related

```
books = Book.objects.all()

for book in books:
    print(book.author.name)
```

Query হবে:

1 query → Book
N query → Author

এটাকে বলে N+1 problem

যদি 100টা book থাকে → 101 query হবে

With select_related

```
books = Book.objects.select_related('author')

for book in books:
    print(book.author.name)
```

Query হবে:

1 query (JOIN)

SQL example:

```
SELECT book.title, author.name
FROM book
JOIN author ON book.author_id = author.id
```

2 prefetch_related

prefetch_related ব্যবহার করা হয়:

- ManyToMany
- Reverse ForeignKey

এটা SQL JOIN করে না।

এটা:

1. multiple query করে
2. Python এ combine করে

Example

```
class Author(models.Model):
    name = models.CharField(max_length=100)

class Book(models.Model):
    title = models.CharField(max_length=100)
    authors = models.ManyToManyField(Author)
```

Use

```
books = Book.objects.prefetch_related('authors')
```

Query হবে:

1 query → books
1 query → authors

তারপর Django Python memory তে relation map করে।

3 Prefetch (Advanced)

Prefetch হলো prefetch_related এর advanced version।

এটা ব্যবহার করা হয় যখন:

- filter করতে হবে
- custom queryset লাগবে
- alias দিতে হবে

Example

```
from django.db.models import Prefetch

books = Book.objects.prefetch_related(
    Prefetch(
        'authors',
        queryset=Author.objects.filter(name__startswith='A')
    )
)
```

এখন শুধু A দিয়ে শুরু হওয়া author আসবে।

Alias ব্যবহার

```
books = Book.objects.prefetch_related(
    Prefetch(
        'authors',
        queryset=Author.objects.filter(name__startswith='A'),
        to_attr='a_authors'
    )
)

for book in books:
    print(book.a_authors)
```

4 Comparison

Feature	select_related	prefetch_related
Relation	FK / OneToOne	ManyToMany / Reverse FK
Query	1 query	2+ query
Method	SQL JOIN	Python combine
Performance	Small relation fast	Large relation better

5 কখন কোনটা ব্যবহার করবেন

ForeignKey / OneToOne

```
Order.objects.select_related('user')
```

ManyToMany / Reverse FK

```
Product.objects.prefetch_related('images')
```

6 Real E-commerce Example

আপনি যেহেতু Django E-commerce project করছেন, এই optimisation খুব দরকার।

Model

```
class Product(models.Model):
    name = models.CharField(max_length=100)

class ProductImage(models.Model):
    product = models.ForeignKey(Product, on_delete=models.CASCADE)
    image = models.ImageField()
```

Without optimization

```
products = Product.objects.all()

for p in products:
    images = p.productimage_set.all()
```

Query হবে:

1 → products
N → product images

With optimization

```
products = Product.objects.prefetch_related('productimage_set')
```

Query হবে:

1 → products
1 → images

7 Mixed Optimization

একসাথে ব্যবহার করা যায়।

```
orders = Order.objects.select_related('user').prefetch_related('items')
```

select_related → SQL JOIN
prefetch_related → Python merge
Prefetch → custom control

আমি Django ORM Query Optimization এর 10টা advanced example (real project level) দেখাতে পারি যেমন:

- select_related multiple level
- prefetch_related nested
- Prefetch + filter
- Prefetch + annotate
- Prefetch + to_attr

Django ORM Optimization – Advanced Examples

1 Multiple select_related

একাধিক ForeignKey একসাথে load করা।

Model

```
class User(models.Model):
    name = models.CharField(max_length=100)

class Order(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)

class Payment(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE)
```

Query select_ related

```
payments = Payment.objects.select_related('order__user')
```

Query without select_ related

```
for p in payments:
    print(p.order.user.name)
```

এখন একটাই query হবে।

2 Multiple prefetch_related

একাধিক ManyToMany বা reverse relation।

```
products = Product.objects.prefetch_related(
    'productimage_set',
    'review_set'
)
```

Query হবে:

```
1 → products
1 → images
1 → reviews
```

3 Nested Prefetch

Relation এর ভিতরের relation prefetch করা।

Model

```
Order
 └── items
     └── product
```

Query

```
orders = Order.objects.prefetch_related(
    'items__product'
)
```

এখন Django items এবং product একসাথে load করবে।

4 Prefetch with Filter

```
from django.db.models import Prefetch

products = Product.objects.prefetch_related(
    Prefetch(
        'review_set',
        queryset=Review.objects.filter(rating__gte=4)
    )
)
```

এখন শুধু ৪☆+ review আসবে।

5 Prefetch with to_attr

Custom attribute এ data save করা।

```
products = Product.objects.prefetch_related(
    Prefetch(
        'review_set',
        queryset=Review.objects.filter(rating__gte=4),
        to_attr='good_reviews'
    )
)

for p in products:
    print(p.good_reviews)
```

6 select_related + prefetch_related একসাথে

Real project এ সবচেয়ে বেশি ব্যবহার হয়।

```
orders = Order.objects.select_related(
    'user'
).prefetch_related(
    'items'
)
```

Query:

1 → orders + user (JOIN)
1 → items

7 Aggregation Optimization

```
from django.db.models import Count

products = Product.objects.annotate(
    review_count=Count('review')
)
```

এখন:

product.review_count

extra query লাগবে না।

8 only() ব্যবহার

Only প্রয়োজনীয় field load করা।

products = Product.objects.only('id', 'name')

এতে memory কম লাগে।

9 defer() ব্যবহার

কিছু field skip করা।

products = Product.objects.defer('description')

Large text field load হবে না।

10 values() Optimization

Object না এনে dictionary আনা।

products = Product.objects.values('id', 'name', 'price')

Example result:

```
[
 {'id':1,'name':'Laptop','price':50000},
 {'id':2,'name':'Phone','price':20000}
]
```

Real E-commerce Query Example

ধরেন:

```
Product
 ├── category
```

```
├── images
└── reviews
```

Optimized query:

```
products = Product.objects.select_related(
    'category'
).prefetch_related(
    'productimage_set',
    'review_set'
)
```

Query হবে:

```
1 → products + category
1 → images
1 → reviews
```

Production Tips

সবসময় use করবেন যখন:

✔ List page
✔ Dashboard
✔ API response
✔ Template loop

Example:

```
{% for product in products %}
{{ product.name }}

{% for img in product.productimage_set.all %}
<img src="{{ img.image.url }}">
{% endfor %}

{% endfor %}
```

Optimization না করলে 100+ query হয়ে যাবে।

Short Rule

ForeignKey / OneToOne → select_related
ManyToMany / Reverse FK → prefetch_related
Custom filter → Prefetch

আপনি চাইলে আমি আরও দেখাতে পারি:

Django ORM Optimization (Pro Level)

- **Subquery**
- **Exists**
- **OuterRef**
- **bulk_create**
- **bulk_update**
- **F expression**
- **database indexing**

এগুলো জানলে আপনার Django skill senior level হয়ে যাবে।

Django ORM Advanced Optimization (Pro Level)

1 F() Expression

F() ব্যবহার করা হয় database level calculation করার জন্য।

Example: product stock কমানো

```
from django.db.models import F

Product.objects.filter(id=1).update(
    stock=F('stock') - 1
)
```

✔ Python এ data load হয় না
✔ Direct database এ update হয়
✔ Fast

2 Subquery

একটা query এর ভিতরে আরেকটা query ব্যবহার করা।

Example

ধরেন প্রতিটি product এর latest review rating আনতে হবে।

```
from django.db.models import Subquery, OuterRef

latest_review = Review.objects.filter(
    product=OuterRef('pk')
).order_by('-created_at')

products = Product.objects.annotate(
    latest_rating=Subquery(latest_review.values('rating')[:1])
)
```

এখন:

```
for p in products:
    print(p.latest_rating)
```

3 Exists

Check করার জন্য relation আছে কিনা।

Example: product এর review আছে কিনা।

```
from django.db.models import Exists, OuterRef

reviews = Review.objects.filter(
    product=OuterRef('pk')
)

products = Product.objects.annotate(
    has_review=Exists(reviews)
)
```

Result:

```
True
False
```

4 bulk_create

একসাথে অনেক object create করা।

Wrong

```
for i in range(1000):
    Product.objects.create(name="Product")
```

এতে 1000 query হবে।

✔ Correct

```
products = [
    Product(name=f"Product {i}")
    for i in range(1000)
]

Product.objects.bulk_create(products)
```

এতে 1 query হবে।

5 bulk_update

একসাথে update।

```
products = Product.objects.all()

for p in products:
    p.price += 10

Product.objects.bulk_update(products, ['price'])
```

6 values() vs values_list()

values()

```
Product.objects.values('id', 'name')
```

Output:

```
{'id': 1, 'name': 'Phone'}
```

values_list()

Product.objects.values_list('name', flat=True)

Output:

['Phone', 'Laptop']

7 annotate() Aggregation

Example: product এর review count।

```
from django.db.models import Count

products = Product.objects.annotate(
    review_count=Count('review')
)
```

8 Database Index

Large table হলে index খুব গুরুত্বপূর্ণ।

```
class Product(models.Model):
    name = models.CharField(max_length=100, db_index=True)
```

বা

```
class Meta:
    indexes = [
        models.Index(fields=['name'])
    ]
```

9 select_for_update

Transaction safe locking।

Example: payment system।

```
from django.db import transaction

with transaction.atomic():
```

```
    product = Product.objects.select_for_update().get(id=1)
    product.stock -= 1
    product.save()
```

Query Count Debug

Django query দেখতে পারবেন:

```
from django.db import connection

print(connection.queries)
```

বা ব্যবহার করতে পারেন:

- **Django Debug Toolbar**

এটা page এ সব SQL query দেখায়।

Real Production Example

E-commerce page:

```
products = Product.objects.select_related(
    'category'
).prefetch_related(
    'productimage_set',
    'review_set'
).annotate(
    review_count=Count('review')
)
```

এতে পাওয়া যাবে:

- **category**
- **images**
- **reviews**
- **review count**

✔ Very few queries
✔ Fast page load

Django ORM Golden Rules

FK / OneToOne → select_related
ManyToMany → prefetch_related
Large insert → bulk_create
Large update → bulk_update
Calculation → F()
Nested query → Subquery
Exist check → Exists